

RAJ ACADEMY · HANDS-ON COURSE FOR SAP ENGINEERS

Governed AI on SAP BTP

Building a read-only order advisor with Claude, MCP and RAG, using the Northwind sample data

Prepared by Rajkumar Rajagobalan

COURSE OUTCOMES

What you will be able to do

Explain why the foundation comes first

Relate identity, connectivity and audit on BTP to the clean-core idea you already know.

Separate facts from policy

Decide what belongs behind a tool call and what belongs in a searchable document store.

Build the advisor end to end

Sign-in, agent, two MCP servers, a policy store in SAP HANA Cloud, and Claude as the model.

Prove that it behaves

Attack it, run 32 evaluation questions, and read the audit trail of every answer.

You need: working knowledge of SAP development, basic Python, and a BTP trial account. No S/4HANA system is used.

Five priorities, in the order to tackle them¹

- 1 Protect the core and platform foundation**
Shared identity, security and operations, so teams build without modifying core systems.
- 2 Integrate the enterprise landscape**
Connect SAP and non-SAP applications, APIs, events and processes.
- 3 Unify business data and context**
Bring business data together for analytics, applications and decisions.
- 4 Automate business processes**
Replace manual handoffs with governed, reusable workflows.
- 5 Enable governed AI innovation**
Build AI applications and agents grounded in approved business context.

What is the business core?

The core is the system of record: it runs the essential transactions and holds the authoritative data. For many SAP customers that is S/4HANA and the systems connected to it.

When poorly designed custom code, uncontrolled integrations or insecure access reach the core, the results are operational disruption, inconsistent data, security exposure and harder upgrades.

Stability here is not negotiable. Every change must be governed.



Why the core needs protection

Upgrade conflicts

Custom changes break, or need rework, whenever SAP ships an update.

Tight coupling

Extensions depend on internal tables and logic that SAP may change.

Support burden

Later teams inherit unfamiliar code long after its authors have left.

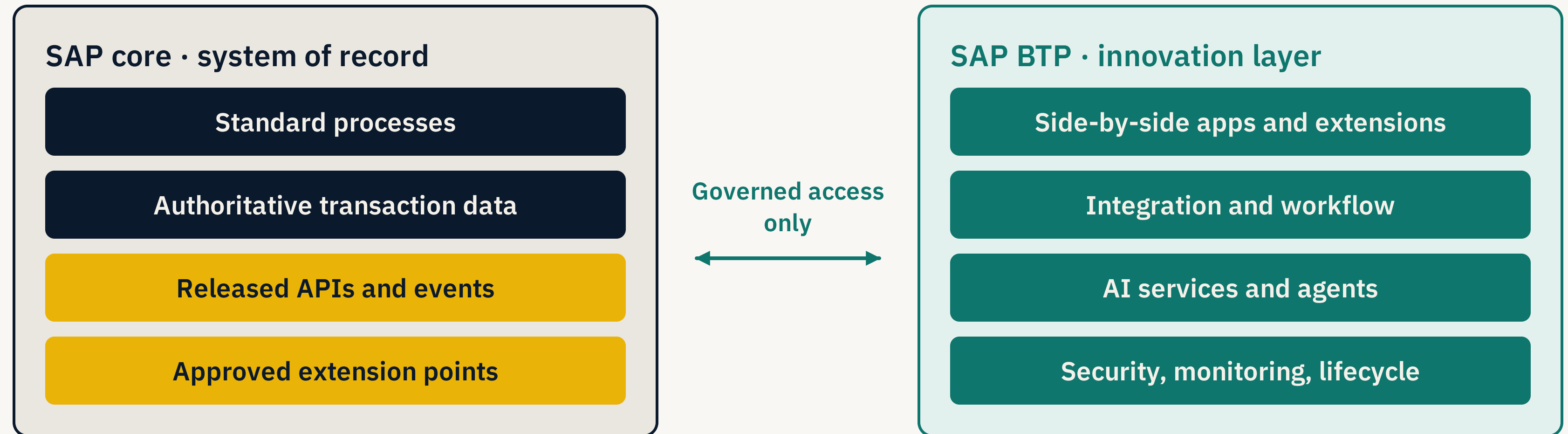
Security exposure

Direct access can bypass the authorization checks the core relies on.

Direct modification solves today's request and creates these four long-term costs.

Goal: solve business problems without giving up core stability or upgrade readiness.

The clean-core approach



The core stays standard and stable. New capability is built beside it and reaches it only through supported interfaces.

What the platform foundation provides

Identity and access

- Role-based authorization, least privilege
- One central sign-in
- A record of who did what

Connectivity and integration

- Managed destinations, no credentials in code
- Released APIs and events
- Standard error handling

Development and lifecycle

- Separate development, test and production
- Version control and automated tests
- Controlled release and rollback

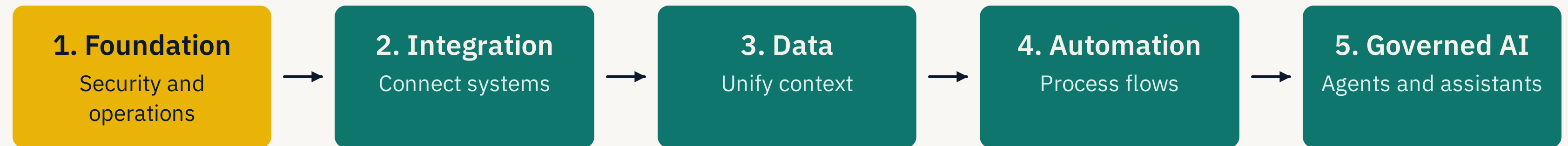
Operations and monitoring

- Health and performance monitoring
- Central logs and tracing
- Alerts and incident response

Without these controls, innovation creates unmanaged risk. The labs use every one of the first two groups.

FROM PRIORITY 1 TO PRIORITY 5

Why AI raises the stakes



An AI agent that reads business data is the newest kind of extension. Without the foundation, it is an uncontrolled one.

Which identity is asking?

In our lab: the signed-in employee, never a shared technical user.

Which data may it read?

Only the rows and documents that this employee may already see.

Which actions may it take?

None. The assistant is read-only by design.

PART 2

The problem and our choice

One concrete business question, three feasible ways to answer it, and the one we will build.

PROBLEM STATEMENT

The question we want answered safely

A SALES EMPLOYEE ASKS

Can order 11019 be expedited?

Northwind Traders is our fictional specialty-food wholesaler.
The data is the Northwind sample database.

A GOOD ANSWER NEEDS THREE THINGS

Live facts

Required date, carrier, freight, and stock on hand for every line.

Policy

When an expedite is allowed, what it costs, and who must approve it.

Limits

Only what this employee may see. The assistant advises; it never approves.

No single source holds all three, and a model on its own will invent the first two.

PROBLEM STATEMENT

Two kinds of knowledge, two access paths

	Live facts	Policy knowledge
Example	Has order 11019 shipped? Is it in stock?	When may an order be expedited?
Changes	Every minute	A few times a year
Lives in	Northwind tables in SAP HANA Cloud	Policy documents, stored as embeddings
Reached through	A tool call over MCP	Retrieval (RAG)
If swapped	Documents quote numbers that are already stale	Rules are hard-coded into tools and hard to change

This split is the central design decision of the course. Every lab builds one side of it, and the capstone joins them.

THREE FEASIBLE SOLUTIONS

Options, with strengths and weaknesses

OPTION A

Use SAP's built-in assistant¹

Configure, do not build.

STRENGTHS

- Least to build and run
- Supported and governed by SAP

WEAKNESSES

- Little control over retrieval and tools
- Hides the mechanics we want to learn

OPTION B

Build outside BTP

BTP only signs the user in.

STRENGTHS

- Widest choice of open-source tooling
- Fast to prototype

WEAKNESSES

- User identity must be carried to a second platform
- Documents and audit trail split across two places

OPTION C · OUR CHOICE

Build side by side on BTP

Agent, tools and RAG all on BTP.

STRENGTHS

- One platform for identity, data and audit
- Reuses SAP skills: CAP, HANA, destinations

WEAKNESSES

- More setup before the first answer
- Fewer ready-made parts

Build side by side on BTP, read-only

Option C keeps identity, data and audit on one platform, and it shows every moving part. That makes it the best teaching vehicle and a sound production pattern.

We keep it simple by keeping it read-only. A wrong answer can mislead; it cannot change an order.

We will build

- A chat page behind BTP sign-in
- An agent that plans, retrieves and checks
- Two read-only MCP servers
- A policy store in SAP HANA Cloud
- An audit trail of every answer

We will not build

- Anything that writes to orders or stock
- Approvals, or messages to customers
- Free-form database or web access for the model
- A connection to S/4HANA

Two tracks, decided by your BTP account

Need	Account with SAP AI Core ¹	BTP trial account
Model access	Generative AI Hub, which brokers the call to Claude	Claude called directly; the key is held in the Destination service
Embeddings	VECTOR_EMBEDDING ² , or an embedding model from the hub	SAP HANA Cloud's built-in VECTOR_EMBEDDING function ²
Audit trail	SAP Audit Log service where entitled, else the audit table	An audit table in SAP HANA Cloud, plus application logs
Guardrails	The hub's masking and filters, plus checks in the agent	Checks written by hand in the agent workflow

Only the model client differs between the two tracks. The agent, both servers, the documents and the labs are identical.

Trial limits³: 90 days, 4 gigabytes of application memory, and applications stop every night and must be restarted.

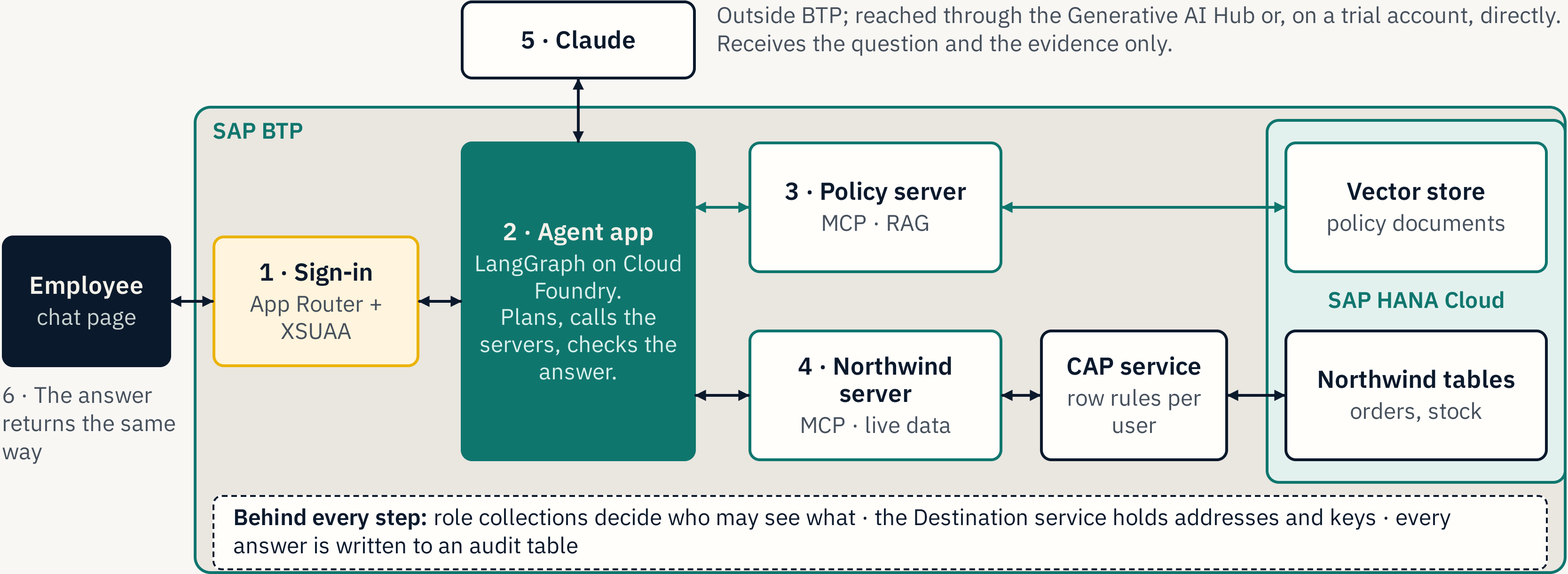
PART 3

The solution

The architecture, the path one question takes through it, and the claims we will put to the test.

ARCHITECTURE

Client, BTP sign-in, agent on BTP, RAG on BTP



One question, six steps

Step	Name	What happens
1	Sign in	The App Router sends the employee to XSUAA, which returns a signed token.
2	Ask	The question and the token reach the agent app.
3	Find policy	The Policy server searches the vector store, filtered by the employee's audience.
4	Get facts	The Northwind server reads orders and stock through the CAP service, filtered by role.
5	Draft	The agent sends the question and the evidence, and nothing else, to Claude.
6	Check and answer	The agent checks citations and limits, writes the audit entry, then replies.

Three rules the architecture encodes

1

The client talks only to the agent

It never reaches the documents, the database or the model directly.

2

Access is decided where the data lives

The Policy server filters documents by audience. The CAP service filters rows by user.

3

Claude explains; it does not act

Read-only tools, no approvals, and a policy source cited in every answer.

Signing in proves who the user is. It does not decide what they may see. Rule 2 does that.

Five hypotheses we will test

	Hypothesis	Tested in
H1	Without grounding, the model invents order facts.	Lab 1
H2	Tools for facts plus retrieval for policy beat either one alone.	Labs 2, 3 and 5
H3	Access decided at the data layer holds even when the prompt is attacked.	Labs 4 and 6
H4	Metadata filters keep superseded and unapproved documents out of answers.	Lab 6
H5	The assistant says that it does not know when the evidence is missing.	Labs 5 and 7

We state these before building, so that the labs are experiments and not demonstrations. We return to them at the end.

PART 4

Technologies, labs and evidence

What we will learn, in the order a request meets it, and how we will know that it works.

TECHNOLOGIES

What we are going to learn

01 Cloud Foundry Runs our five applications	02 App Router + XSUAA Sign-in and signed tokens	03 LangGraph Agent as explicit steps	04 LangChain Model and retriever parts
05 MCP Standard tool interface	06 MCP Python SDK Builds the two servers	07 CAP + OData Data service, row rules	08 HANA vector engine Stores and searches policy
09 Embeddings Turn text into vectors	10 RAG Answer from passages	11 Claude Reasoning and drafting	12 Destination service Keeps keys out of code

Ordered along the path a request takes. Each has its own page in the appendix.

The APIs we will call, and why

API	Called by	Purpose
Claude Messages API, through the hub or directly	Agent app	Reasoning, choosing a tool, drafting the answer
SAP HANA Cloud SQL, with VECTOR_EMBEDDING	Policy server, loader	Store and search policy vectors; create embeddings
Our CAP OData service	Northwind server	Orders, stock and suppliers, filtered by role
MCP: tools/list and tools/call	Agent app	One standard way to discover and call tools
XSUAA, using OAuth 2.0	Every application	Sign-in, token checks, roles and attributes
Destination service API	Agent and servers	Look up addresses and keys at run time
OpenAI embeddings API	Loader, as a fallback	Only if VECTOR_EMBEDDING is unavailable; Claude makes no embeddings

Libraries, not web APIs: LangChain, LangGraph, langchain-hana, langchain-mcp-adapters, pdfplumber.

Eight labs, one element each

Lab	Element taught	What you build
0	Platform foundation	Trial account, roles, SAP HANA Cloud, Northwind loaded, destinations
1	Why grounding matters	A bare Claude call; watch it invent an order status
2	RAG	Load the 11 policy documents; retrieve with citations and filters
3	MCP	One tool, get_order_status, over the CAP service
4	Identity	Two users, the same question, different rows and documents
5	Orchestration	Both servers in one LangGraph workflow, with an answer check
6	Adversarial testing	A superseded policy, a conflicting memo, an injected email
7	Evaluation and operations	Run the 32 questions; read the audit table; count the cost

The lab data

830

Northwind orders

with 77 products and 91 customers, loaded into SAP HANA Cloud

11

policy documents

7 text files and 4 PDF files: policies, account notes, a runbook, a glossary

3

robustness documents

a superseded policy, a conflicting memo, an email with an injection attempt

32

evaluation questions

each with its expected tool calls, sources and answer

Design rule: numbers live in tables, rules live in documents. One memo breaks the rule on purpose, so that you can catch it.

All Northwind dates are shifted forward by 28 years. The labs use a fixed today of Thursday 7 May 2026.

EVIDENCE

How we will know that it works

FOUR CHECKS ON EVERY ANSWER

Facts match

The figures equal the expected answer.

Sources cited

The expected policy documents are named.

Tools called

The expected tool calls were made.

Nothing forbidden

No guess, restricted figure, approval or injected text.

THE 32 QUESTIONS

Kind of question	Count
Tool only	3
Retrieval only	7
Tool and retrieval combined	12
Abstain, or refuse an action	3
Access control and stale data	3
Robustness	4

The hypotheses, revisited

	Hypothesis	Result
H1	Without grounding, the model invents order facts.	[fill in after Lab 7]
H2	Tools for facts plus retrieval for policy beat either one alone.	[fill in after Lab 7]
H3	Access decided at the data layer holds even when the prompt is attacked.	[fill in after Lab 7]
H4	Metadata filters keep superseded and unapproved documents out of answers.	[fill in after Lab 7]
H5	The assistant says that it does not know when the evidence is missing.	[fill in after Lab 7]

Record the evidence for each: the evaluation questions that passed or failed, and what you changed to fix a failure.

AFTER THE COURSE

From Northwind to your SAP landscape

In the lab	In your SAP landscape
Northwind tables behind a CAP service	Released S/4HANA APIs behind the same destination
Row rules written in CAP	S/4HANA authorizations, with the user's identity passed through ¹
Claude through the hub, or directly on a trial	Generative AI Hub in SAP AI Core, for everyone
Audit table in SAP HANA Cloud	SAP Audit Log service
One trial subaccount	Separate development, test and production subaccounts

What stays the same: the agent, the two MCP interfaces, the documents, and the evaluation set.

KEY TAKEAWAY

Governed AI on BTP is clean-core engineering applied to a new kind of extension.

Supported interfaces to the data. Access decided where the data lives. Evidence behind every answer.

APPENDIX

The technologies, one page each

Every page answers the same questions: what it is, why we need it here, where it sits, the call you will write, and the common mistake.

Cloud Foundry

WHAT IT IS

The application runtime of BTP. You push source code; the platform builds it, runs it and gives it an address.

WHY WE NEED IT HERE

It hosts our five applications: the App Router, the agent, the two MCP servers and the CAP service.

COMMON MISTAKE

Forgetting the trial limits. Applications stop every night, and all five must fit in 4 gigabytes of memory.

WHERE IT SITS

Underneath everything inside the BTP boundary.

THE CALL YOU WILL WRITE (SKETCH)

```
cf push agent-app -m 512M
cf logs agent-app --recent
cf restart agent-app
```

App Router and XSUAA

WHAT IT IS

The App Router is the single entry point. XSUAA signs the user in and issues a signed token, a JWT, that carries roles and attributes.

WHY WE NEED IT HERE

Every later check depends on knowing who is asking. The token travels with the question to the agent and the servers.

COMMON MISTAKE

Treating sign-in as authorization. A valid token does not decide which rows or documents a user may see.

WHERE IT SITS

Step 1, between the employee and the agent.

THE CALL YOU WILL WRITE (SKETCH)

```
from sap import xssec
ctx = xssec.create_security_context(token, uaa)
if not ctx.check_scope("advisor.read"):
    deny()
office = ctx.get_attribute("office")
```

LangGraph

WHAT IT IS

A Python library for writing an agent as an explicit graph of steps that share one state.

WHY WE NEED IT HERE

Our safety steps become visible, testable nodes: authorize, retrieve, call a tool, draft, check the answer.

COMMON MISTAKE

Calling every step deterministic. Rule checks are. Model-based checks, such as spotting an injection, are not, so they cannot be the last line of defence.

WHERE IT SITS

Step 2. It is the agent app.

THE CALL YOU WILL WRITE (SKETCH)

```
g = StateGraph(State)
g.add_node("retrieve", retrieve)
g.add_node("draft", draft)
g.add_node("check", check_answer)
g.add_edge("retrieve", "draft")
app = g.compile()
```

LangChain

WHAT IT IS

A Python library of building blocks: chat model clients, retrievers, document loaders and tool wrappers.

WHY WE NEED IT HERE

It gives us the Claude client, the SAP HANA Cloud retriever and tool binding without hand-written plumbing.

COMMON MISTAKE

Letting a ready-made agent run free. We use the parts and keep the control flow in LangGraph.

WHERE IT SITS

Inside the agent app and the Policy server.

THE CALL YOU WILL WRITE (SKETCH)

```
llm = ChatAnthropic(model=MODEL_ID)
llm = llm.bind_tools(tools)
reply = llm.invoke(messages)
```

MCP

WHAT IT IS

An open protocol for offering tools to AI applications. A server lists its tools with typed inputs; a client calls them.

WHY WE NEED IT HERE

One governed, reusable interface. The same two servers could serve a second assistant later without change.

COMMON MISTAKE

Enforcing limits in the prompt. Limits belong inside the server: a fixed tool list, input checks and a user check.

WHERE IT SITS

Steps 3 and 4, between the agent and the two servers.

THE CALL YOU WILL WRITE (SKETCH)

```
→ tools/list
← get_order_status(order_id: int)
→ tools/call get_order_status
    {"order_id": 11019}
← {"status": "not shipped", ...}
```

MCP Python SDK and adapters

WHAT IT IS

FastMCP, from the MCP Python SDK, turns a Python function into an MCP tool. The langchain-mcp-adapters package turns MCP tools into LangChain tools.

WHY WE NEED IT HERE

A server is about twenty lines, and the agent discovers the tools when it starts.

COMMON MISTAKE

Exposing generic tools such as run any query. Name each tool after one business question.

WHERE IT SITS

Inside both MCP servers, and in the agent as the client.

THE CALL YOU WILL WRITE (SKETCH)

```
mcp = FastMCP("northwind")

@mcp.tool()
def get_order_status(order_id: int) → dict:
    return read_order(order_id)

mcp.run(transport="streamable-http")
```

CAP and OData

WHAT IT IS

SAP's framework for data services. You model entities in CDS and receive an OData service, with authorization rules declared beside the model.

WHY WE NEED IT HERE

It stands in for a released SAP API, and it filters rows for each user before anything reaches the MCP server.

COMMON MISTAKE

Copying the row rules into the MCP server. Keep them in one place, the data service.

WHERE IT SITS

Step 4, between the Northwind server and the tables.

THE CALL YOU WILL WRITE (SKETCH)

```
entity Orders @(restrict: [{  
  grant: 'READ',  
  where: 'ShipCountry = $user.country'  
}]) as projection on db.Orders;  
  
GET /odata/v4/northwind/Orders(11019)
```

SAP HANA Cloud vector engine

WHAT IT IS

A vector column type and similarity functions inside SAP HANA Cloud. The langchain-hana package wraps it as a vector store.

WHY WE NEED IT HERE

Policy passages and their metadata sit beside the Northwind tables: one database to secure, back up and audit.

COMMON MISTAKE

Filtering after the search. Apply the status and audience filters inside the query, so restricted text never leaves the store.

WHERE IT SITS

Step 3, behind the Policy server.

THE CALL YOU WILL WRITE (SKETCH)

```
db = HanaDB(connection=conn,
             embedding=emb,
             table_name="POLICY_CHUNKS")
hits = db.similarity_search(
    question, k=4,
    filter={"status": "current"})
```

Embeddings

WHAT IT IS

Lists of numbers that represent meaning. Texts with similar meaning get vectors that lie close together.

WHY WE NEED IT HERE

Claude does not create embeddings, so a second model is needed. SAP HANA Cloud's built-in function keeps the text inside the database.

COMMON MISTAKE

Using one model for documents and another for questions. The vectors are then not comparable. Re-embed everything when the model changes.

WHERE IT SITS

In the loader, and in the Policy server for each question.

THE CALL YOU WILL WRITE (SKETCH)

```
SELECT VECTOR_EMBEDDING(  
    :question, 'QUERY', :model_id)  
FROM DUMMY;
```

RAG

WHAT IT IS

At question time, retrieve the most relevant passages and give them to the model with an instruction to answer only from them and to cite them.

WHY WE NEED IT HERE

Policies change. Retrieval keeps answers current and traceable to a document version, with no retraining.

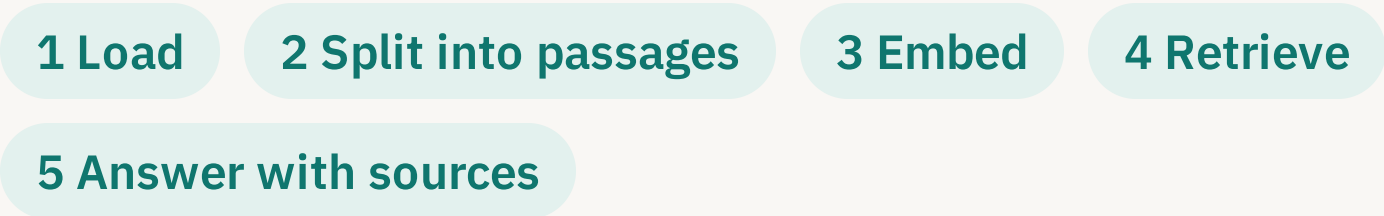
COMMON MISTAKE

Embedding the database tables. Structured facts belong behind tools; RAG is for documents.

WHERE IT SITS

Step 3. The Policy server and the vector store together.

THE FIVE STAGES



The loader runs stages 1 to 3 once per document version. Stages 4 and 5 run for every question.

Claude and the Messages API

WHAT IT IS

Anthropic's API for Claude. You send system instructions, the conversation and tool definitions; you receive text or a request to call a tool.

WHY WE NEED IT HERE

Claude chooses which tool to call, reads the evidence and drafts the cited answer. It holds no credentials and reaches no data itself.

COMMON MISTAKE

Sending more than the answer needs. The request leaves BTP, so send only the retrieved passages and the tool results.

WHERE IT SITS

Step 5, outside the BTP boundary. Reached through the Generative AI Hub where your account has it.

THE CALL YOU WILL WRITE (SKETCH)

```
client.messages.create(  
    model=MODEL_ID, max_tokens=1024,  
    system=RULES,  
    messages=msgs,  
    tools=tool_specs)
```

Destination service

WHAT IT IS

A BTP service that stores the address and credentials of a remote system under a name.

WHY WE NEED IT HERE

It holds the address of the CAP service, and the address and key of the Claude API. Nothing secret sits in code.

COMMON MISTAKE

Putting an API key in an environment file that is then committed to the source repository.

WHERE IT SITS

Behind every outbound call from the agent and the servers.

THE CALL YOU WILL WRITE (SKETCH)

```
GET /destination-configuration/v1/  
    destinations/CLAUDE_API  
Authorization: Bearer <service token>
```

Audit trail

WHAT IT IS

One record for every answer: who asked, which documents were retrieved, which tools ran, and the outcome.

WHY WE NEED IT HERE

It lets a reviewer reconstruct any answer, and it supplies the evidence for Lab 7. In production, the SAP Audit Log service takes this role.

COMMON MISTAKE

Logging whole prompts with business data in them. Store identifiers, not content.

WHERE IT SITS

Written by the agent at step 6, into SAP HANA Cloud.

THE CALL YOU WILL WRITE (SKETCH)

```
INSERT INTO AUDIT_EVENTS
  (AT, USER_ID, QUESTION_ID,
   DOC_IDS, TOOLS, OUTCOME)
VALUES (...);
```

Generative AI Hub

WHAT IT IS

Part of SAP AI Core. It gives governed access to models from several providers, Claude among them, with optional data masking and content filtering.

WHY WE NEED IT HERE

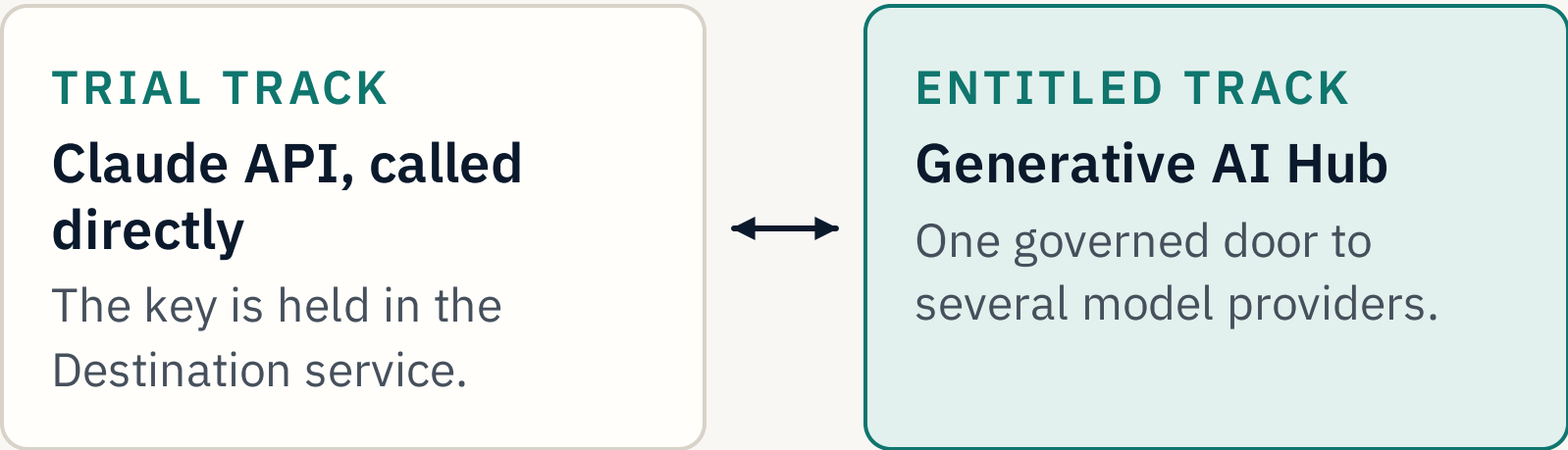
If your BTP account includes SAP AI Core with the extended plan, use it for step 5: the model call then stays under SAP's governance. Trial accounts do not offer it.

COMMON MISTAKE

Assuming the track changes the design. Only the model client changes; the agent, servers and documents stay as they are.

WHERE IT SITS

Step 5, inside the BTP boundary, in front of Claude.



To check: BTP cockpit, Entitlements, SAP AI Core, plan extended. SAP also runs a separate 30-day trial of the hub.

Glossary of acronyms

Term	Meaning
BTP	SAP Business Technology Platform
AI	artificial intelligence
RAG	retrieval-augmented generation
MCP	Model Context Protocol
API	application programming interface
ERP	enterprise resource planning
S/4HANA	SAP's ERP suite
HANA	SAP's in-memory database
XSUAA	SAP Authorization and Trust Management service
JWT	JSON Web Token, a signed proof of identity

Term	Meaning
OAuth	open standard for delegated sign-in
CAP	SAP Cloud Application Programming Model
CDS	Core Data Services, CAP's modelling language
OData	Open Data Protocol
SQL	Structured Query Language
SDK	software development kit
CI/CD	continuous integration and delivery
PDF	Portable Document Format
GB	gigabytes